

# Dual-Stream YOLOv8 with Cross-Reference Fusion for SMT Component Defect Detection on PCB Production Lines

Vuong Thanh Tung

Faculty of Electronics and Telecommunications  
University of Engineering and Technology, VNU  
Hanoi, Vietnam  
**[TODO: email]**

Hoang Van Manh

Faculty of Electronics and Telecommunications  
University of Engineering and Technology, VNU  
Hanoi, Vietnam  
**[TODO: email]**

**Abstract**—Automated visual inspection (AVI) of surface-mount-technology (SMT) printed circuit boards is a constrained instance of object detection in which a *golden reference image* of a known-good board is always available at inference time. State-of-the-art detectors, however, ignore this information and process the captured frame in isolation, forcing the network to implicitly learn the expected component layout from scratch and making it brittle on the small, class-imbalanced datasets typical of real production cells. We propose DS-YOLO (Dual-Stream YOLO), a Siamese extension of YOLOv8s that routes the captured frame and the golden reference through a shared backbone in parallel; their multi-scale feature maps at  $P_3, P_4, P_5$  are fused by a learnable *Cross-Reference Fusion Module* (CRFM) before the PANet neck and detection head. Because fusion operates at feature scale (stride 8–32 px), DS-YOLO is robust to sub-pixel fixture drift without any pixel-level alignment pre-processing. The CRFM adds only  $\sim 173$  K parameters and is initialised so the network starts as the YOLOv8s baseline, keeping the pretrained prior fully intact. DS-YOLO is compared against an RGB-only baseline and two reference-conditioning variants—input-level concatenation (Stack6) and parameter-free feature subtraction (F-Sub)—on an in-house dataset of 75 raw captures (211 after Roboflow augmentation) from a single SMT board layout, covering four component types, under a unified training and evaluation protocol. DS-YOLO improves mAP@0.5 over the RGB-only baseline while adding less than **[TODO:  $\delta t$ ]** ms latency, with its clearest advantage in the low-data regime. **[TODO: Headline numbers to be filled after the unified multi-seed re-run.]** The detector is deployed on a working SMT cell where a PC-side vision server returns OK/NG verdicts to a Mitsubishi MELSEC-Q PLC via OPC UA, and the PLC orchestrates two articulated robots over CC-Link to physically sort defective boards.

**Index Terms**—Automated visual inspection, object detection, YOLOv8, dual-stream networks, reference-guided detection, cross-reference fusion, SMT, PCB, OPC UA.

## I. INTRODUCTION

Surface-mount-technology (SMT) assembly is the standard process by which modern printed circuit boards (PCBs) are populated. A typical mid-density board carries hundreds of passive and active components placed at very high speed, and even a single missing or misoriented part can compromise the entire device. Visual inspection of every produced board is therefore mandatory, but manual inspection is slow, expensive

in labor and inherently subjective. Commercial Automated Optical Inspection (AOI) machines deliver adequate accuracy at the price of a high acquisition cost and limited adaptability to fast-changing product mixes—a poor match for low-volume, high-mix electronics manufacturing.

A natural alternative is to combine an industrial camera with a deep object detector trained on the components of interest. The YOLO family of detectors [1], [2] is particularly attractive in this setting thanks to its accuracy/throughput trade-off, and several recent works have applied it to PCB defect detection [3]–[5]. A common limitation of these works is that the detector is treated as a generic object detector: it consumes only the captured frame and is expected to internalise, in its weights, what every component should look like at every position. This is wasteful, because in an AVI cell a *golden reference image* of a known-good board is always available—it is the very specification of what the line is supposed to produce. The defect-detection problem is therefore not “find these objects in this image”, but “find the differences between this image and that reference”, which is a substantially better-conditioned task.

**Contributions.** Motivated by this observation, we make three contributions.

- We formalise SMT defect detection as a *reference-guided detection* problem and systematically compare how the golden reference  $I_g$  should be fused into a YOLOv8s detector: input-level concatenation (Stack6), parameter-free feature subtraction (F-Sub) and the proposed learnable feature-level fusion (DS-YOLO), all against an RGB-only baseline under one common training and evaluation protocol.
- We propose **DS-YOLO**, a Siamese dual-stream extension of YOLOv8s that processes the captured frame and the golden reference through a shared backbone and fuses their multi-scale features at  $P_3, P_4, P_5$  via a learnable Cross-Reference Fusion Module (CRFM). The CRFM adds only  $\sim 173$  K parameters ( $\sim 1.8\%$  of the backbone) and is initialised to the identity so that the pretrained

YOLOv8 prior is fully preserved at the start of training.

- We deploy the resulting system on a working SMT inspection cell that integrates a Mitsubishi MELSEC-Q PLC, two articulated robots from different vendors and a Basler GigE camera over CC-Link and OPC UA, and we analyse its end-to-end latency and robustness to residual alignment errors.

The remainder of this paper is organised as follows. Section II reviews related work. Section III formalises reference-guided detection and states our hypotheses. Section IV presents DS-YOLO, the CRFM and the five comparison variants. Section V summarises the industrial deployment. Experiments are reported in Section VI and Section VII concludes.

## II. RELATED WORK

*Classical AOI.* Early AOI systems for PCB inspection rely on hand-crafted features such as template matching, normalised cross-correlation and morphological operators [6]. They achieve high precision when the imaging conditions are tightly controlled but degrade rapidly with illumination, color or layout variations and require manual re-tuning for every new board.

*Deep detectors for PCB inspection.* Convolutional detectors — Faster R-CNN, SSD and the YOLO family — have largely replaced classical pipelines for component-level defect detection [1]–[4]. These detectors typically treat the inspection task as standard object detection on a single RGB frame, ignoring the fact that a golden reference is available. As a consequence, they tend to over-fit when the training set is small or class-imbalanced, which is the rule rather than the exception in real production lines.

*Reference- and template-based deep learning.* Several works exploit reference images or template comparison in related domains. Change detection in remote sensing [7] fuses bi-temporal images through Siamese encoders and feature subtraction. Anomaly detection methods such as PaDiM and PatchCore [8], [9] model the distribution of reference patches and flag deviations at inference. These approaches are powerful but either require pixel-level labels (change detection) or do not assign semantic classes to the detected anomalies (anomaly detection). The closest line of work to ours uses reference or template images for PCB inspection — e.g. Siamese segmentation of welding defects [10] and template/test image pairs for bare-board trace defects [5], [11] — but these target either pixel-wise segmentation or bare-board copper-trace defects (open, short, mousebite) and mostly couple the reference at the input or pixel level. Our contribution is to integrate the golden reference as a *learnable, identity-initialised feature-level fusion* (CRFM) inside a one-stage YOLO detector that simultaneously localises and classifies *SMT component-placement* defects, and to deploy it on a real production cell — a combination, to our knowledge, not previously reported.

*Industrial integration.* On the controller side, CC-Link [12] provides deterministic field-level traffic between a PLC and remote stations, while OPC UA (IEC 62541) [13] offers

a vendor-neutral, secure protocol for the PLC-to-supervisory layer. Their combined use is recommended by the RAMI 4.0 reference model and has been reported in several Industry 4.0 case studies [14], [15], but their use as the data plane for a deep-learning vision server has received comparatively less attention in the academic literature.

## III. PROBLEM FORMULATION

Let  $\mathbf{I}_g \in \mathbb{R}^{H \times W \times 3}$  denote a fixed RGB image of a known-good (*golden*) PCB, and let  $\mathbf{I}_c \in \mathbb{R}^{H \times W \times 3}$  denote a frame captured during inspection. A defect detector is a function

$$\mathcal{F} : \mathcal{X} \longrightarrow \{(b_i, c_i, s_i)\}_{i=1}^N, \quad (1)$$

where  $(b_i, c_i, s_i)$  is the bounding box, class label and confidence score of the  $i$ -th detected component and  $N$  is unknown a priori. We distinguish two regimes:

- *Image-only detection* ( $\mathcal{X} = \mathbb{R}^{H \times W \times 3}$ ): the detector observes only the captured frame,  $\mathcal{F}(\mathbf{I}_c)$ . This is the standard YOLO setting.
- *Reference-guided detection* ( $\mathcal{X} = \mathbb{R}^{H \times W \times 3} \times \mathbb{R}^{H \times W \times 3}$ ): the detector observes both the capture and the reference,  $\mathcal{F}(\mathbf{I}_c, \mathbf{I}_g)$ .

Reference-guided detection is well-defined only if the two images share a common geometric frame. In practice, mechanical drift in the conveyor and the fixture introduces a non-trivial planar transformation between  $\mathbf{I}_c$  and  $\mathbf{I}_g$ . We therefore precede the detector by an alignment step that estimates a homography  $\mathbf{H}$  from a set of ORB feature matches and warps the capture into the reference frame:

$$\tilde{\mathbf{I}}_c = \mathcal{W}(\mathbf{I}_c, \mathbf{H}), \quad \mathbf{H} \approx \arg \min_{\mathbf{H}} \rho(\mathcal{M}(\mathbf{I}_c, \mathbf{I}_g), \mathbf{H}), \quad (2)$$

where  $\mathcal{W}$  denotes bilinear warping,  $\mathcal{M}$  the set of ORB feature matches and  $\rho$  a robust RANSAC residual.

We then form a per-pixel absolute-difference map

$$\mathbf{D} = |\tilde{\mathbf{I}}_c - \mathbf{I}_g| \in \mathbb{R}^{H \times W \times 3}, \quad (3)$$

which is large near defect regions and small elsewhere.

*Hypotheses.* We test three hypotheses experimentally:

- H1** Conditioning a YOLOv8 detector on the golden reference (through CRFM’s feature difference) improves mAP@0.5 over the same backbone trained on  $\tilde{\mathbf{I}}_c$  alone, particularly on the minority *NG* class whose defect signature is geometric and is therefore well captured by a reference difference.
- H2** Learnable feature-level fusion (CRFM in DS-YOLO) outperforms both (a) raw feature subtraction (F-Sub, zero extra parameters) and (b) early input-level fusion (Stack6), because the gated residual formulation lets the network decide per-spatial-location how much the reference difference should modulate the capture features.
- H3** The benefit of  $\mathbf{D}$  is largest when the training set is small, because the difference channel exposes the defect location explicitly and reduces what the network has to learn.

#### IV. METHOD: DUAL-STREAM YOLO

Fig. 1 illustrates the proposed DS-YOLO. The pipeline is composed of three modules: (i) reference-aware alignment, (ii) a shared YOLOv8s backbone applied in a Siamese fashion to the capture and the golden reference, and (iii) a Cross-Reference Fusion Module (CRFM) that merges the two feature streams before the YOLOv8 detection head.

##### A. Reference-Aware Alignment

On the production cell the camera and the board fixture are mechanically fixed, so  $\mathbf{I}_c$  and  $\mathbf{I}_g$  are already co-registered and DS-YOLO operates directly on the capture ( $\tilde{\mathbf{I}}_c = \mathbf{I}_c$ ). When this assumption does not hold, an optional pre-alignment estimates a homography  $\mathbf{H}$  from up to  $K=2000$  ORB keypoints (brute-force Hamming matching, Lowe ratio  $\tau=0.75$ ) with RANSAC ( $\sigma=3.0$  px) and warps the capture into the reference frame. The robustness study in Section VI quantifies tolerance to residual registration error  $\sigma_t$  when this co-registration is imperfect.

##### B. Cross-Reference Fusion Module

At each scale  $\ell \in \{P_3, P_4, P_5\}$  the backbone produces a capture feature map  $\mathbf{F}_c^\ell$  and a golden feature map  $\mathbf{F}_g^\ell$  of equal shape. The Cross-Reference Fusion Module (CRFM) computes a learnable difference attention and modulates the capture features:

$$\mathbf{D}^\ell = |\mathbf{F}_c^\ell - \mathbf{F}_g^\ell|, \quad (4)$$

$$\mathbf{M}^\ell = \sigma(g_\ell(\mathbf{D}^\ell)), \quad (5)$$

$$\mathbf{F}_{\text{fused}}^\ell = \mathbf{F}_c^\ell + \alpha_\ell \mathbf{M}^\ell \odot \mathbf{F}_c^\ell, \quad (6)$$

where  $g_\ell$  is a 2-layer  $1 \times 1$  convolution with a  $4 \times$  channel bottleneck,  $\sigma$  is the sigmoid,  $\odot$  is the element-wise product, and  $\alpha_\ell \in \mathbb{R}$  is a learnable scalar initialised to zero. At initialisation  $\mathbf{F}_{\text{fused}}^\ell = \mathbf{F}_c^\ell$ , so DS-YOLO reduces exactly to a YOLOv8s baseline; as training progresses,  $\alpha_\ell$  and  $g_\ell$  learn how much and where to amplify capture features that disagree with the reference.

##### C. DS-YOLO Architecture

The full network composes (i) a single YOLOv8s backbone applied *twice* (once on  $\tilde{\mathbf{I}}_c$ , once on  $\mathbf{I}_g$ , with shared weights), (ii) three CRFM modules at  $P_3, P_4, P_5$ , and (iii) the unmodified YOLOv8s PANet neck and Detect head. The parameter overhead versus the YOLOv8s baseline is  $\sim 173$  K parameters ( $\sim 1.8\%$  of the model) concentrated in the three CRFM gates; the compute overhead is one extra backbone forward pass on the golden image,  $\sim 50\%$  FLOPs at inference time. We initialise the backbone, neck and head from the public Ultralytics `yolo_v8s.pt` checkpoint [2]; the CRFM modules use Kaiming initialisation for  $g_\ell$  and zero for  $\alpha_\ell$ .

##### D. Compared Variants

To isolate the contribution of each design choice we evaluate four configurations on the same YOLOv8s backbone:

- 1) **RGB** (baseline): single-stream YOLOv8s on  $\tilde{\mathbf{I}}_c$ .
- 2) **Stack6**: single-stream YOLOv8s taking the 6-channel concatenation  $[\tilde{\mathbf{I}}_c, \mathbf{I}_g]$  (input-level fusion).
- 3) **F-Sub**: dual-stream backbone with raw feature subtraction  $\mathbf{F}_{\text{fused}}^\ell = \mathbf{F}_c^\ell - \mathbf{F}_g^\ell$  in place of CRFM; zero extra parameters.
- 4) **DS-YOLO** (ours): dual-stream backbone with learnable CRFM as described in Section IV-B.

These span the two design axes of interest: input-level vs. feature-level fusion, and parameter-free (F-Sub) vs. learnable (CRFM) feature fusion. Stack6 requires enlarging the first convolution of the backbone to accept  $C=6$  input channels; the original RGB weights are copied into the first three channels and the golden channels are initialised with the channel-mean of the pretrained weights, as in [2]. Other input-level encodings of the reference (a difference-only channel, or a 4-channel RGB+diff input) are possible but not evaluated in this version.

##### E. Loss Function

We retain the default YOLOv8 loss—a sum of distribution-focal, CIoU and binary cross-entropy terms—with no class re-weighting, so that any improvement is attributable to the input modification rather than to a richer optimisation objective. Implementation details are given in Section VI.

#### V. INDUSTRIAL DEPLOYMENT

We integrate the DS-YOLO detector into a working SMT inspection cell (Fig. 2). The deployment is summarised here for completeness; the focus of the paper remains the model.

**Hardware.:** The cell is built around a Mitsubishi Q13UDVCPULC fitted with a QJ61BT11N CC-Link master and a QD77MS16 simple-motion module driving an MR-J4-20B servo amplifier. Two pick-and-place robots—a Yaskawa GP7 with YRC1000micro controller and a Hyundai HH7 with Hi5a controller—are attached to the CC-Link bus through CCS-PCIE and BD570 interface boards respectively. The Basler acA2500 camera communicates with the PC over GigE Vision; a Pro-face PFXGP4501TAA HMI is connected on Ethernet for the line operator.

**Communication.:** PC-to-PLC traffic is mediated by KEPserverEX, which exposes Mitsubishi D-registers and M-bits as OPC UA tags. The Python vision server subscribes to a `TriggerCapture` bit and writes back the `Verdict` and `InspectionDone` flags after each cycle. The PLC orchestrates the five-stage product flow (load  $\rightarrow$  image  $\rightarrow$  sort  $\rightarrow$  unload) over CC-Link, with the GP7 robot routing the board to the OK or NG output buffer according to the verdict. The CC-Link station map is reported in Table I.

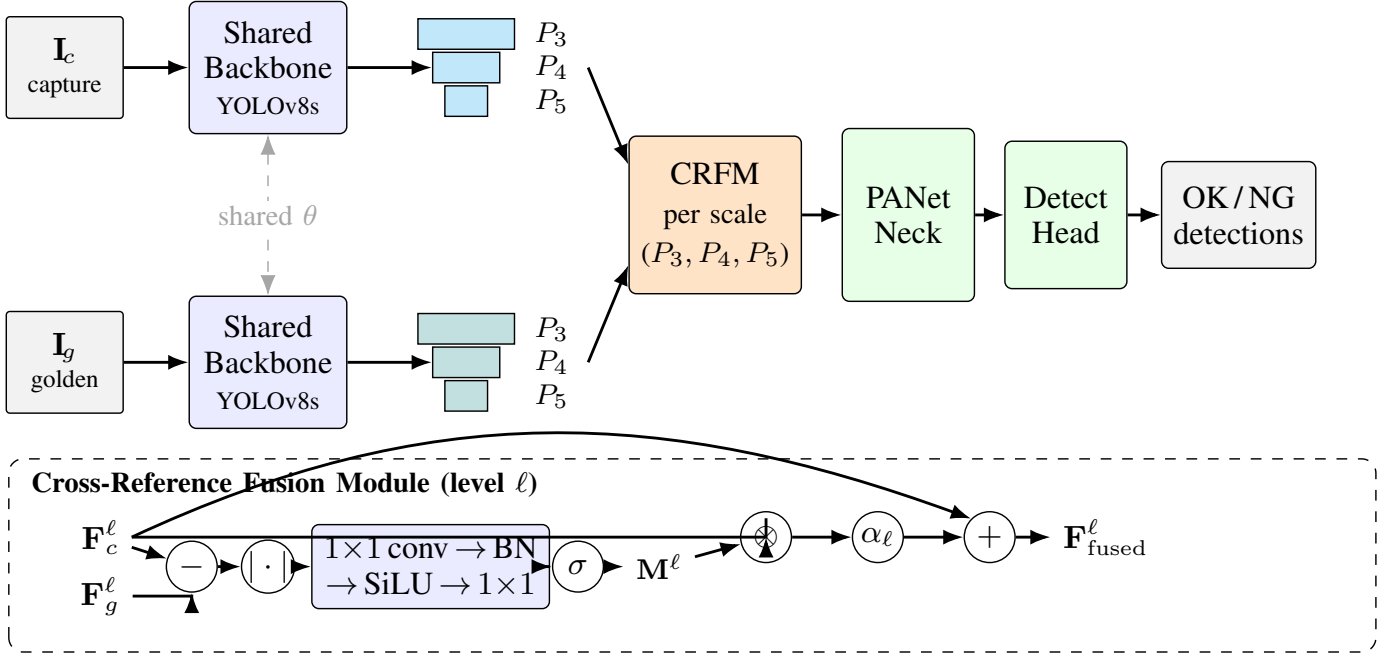


Fig. 1. Architecture of the proposed Dual-Stream YOLO (DS-YOLO). The captured frame  $\mathbf{I}_c$  and the always-available golden reference  $\mathbf{I}_g$  (pre-registered by the fixed camera and fixture) are encoded by a *shared* (Siamese) YOLOv8s backbone. At every pyramid level  $\ell \in \{P_3, P_4, P_5\}$  a Cross-Reference Fusion Module (CRFM) fuses the capture and golden features through a learnable difference-attention gate (inset):  $\mathbf{M}^\ell = \sigma(g_\ell(|\mathbf{F}_c^\ell - \mathbf{F}_g^\ell|))$  and  $\mathbf{F}_{\text{fused}}^\ell = \mathbf{F}_c^\ell + \alpha_\ell \mathbf{M}^\ell \odot \mathbf{F}_c^\ell$ , with  $\alpha_\ell$  initialised to 0 so the network starts *exactly* as the YOLOv8s baseline. The fused feature pyramid feeds the unmodified PANet neck and Detect head, which output per-component OK/NG detections.



Fig. 2. The deployed SMT inspection cell. A Basler GigE camera images each board on the conveyor; a Mitsubishi MELSEC-Q PLC orchestrates the cycle over CC-Link and routes the board to the OK or NG buffer using two articulated robots (a Yaskawa GP7 and a Hyundai HH7).

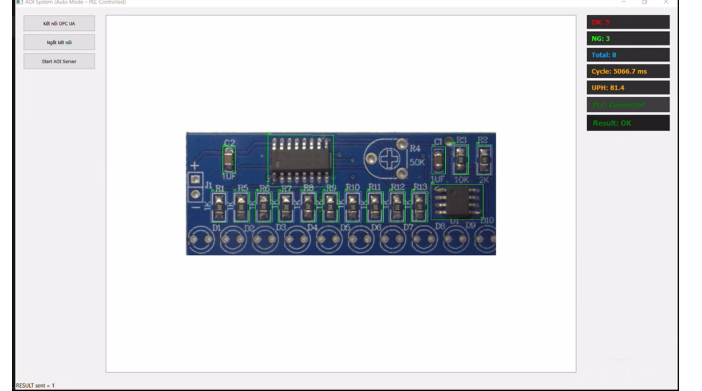


Fig. 3. Operator view of the AOI station in automatic mode. The GUI overlays component-level OK/NG results on the captured board and reports the per-board cycle time and throughput (UPH); the vision server exchanges the trigger and verdict with the PLC over OPC UA.

*Throughput.*: In automatic (PLC-controlled) operation the station sustains a per-board cycle time of  $\approx 5.1$  s (load  $\rightarrow$  image  $\rightarrow$  sort  $\rightarrow$  unload), i.e. a sustained throughput of  $\approx 81$  boards per hour (Fig. 3). This end-to-end cycle is dominated by robot handling; the vision inference itself is only a small fraction of it (Table VIII).

## VI. EXPERIMENTS

### A. Dataset

Images were captured directly on the production cell described in Section V under varying lighting, board place-

ment and component states — all from a single physical board layout. We collected 75 unique raw captures and used Roboflow’s offline augmentation pipeline (flips, HSV jitter, mosaic) to expand the annotated set to 211 images. Each image was annotated manually on Roboflow with component-level bounding boxes covering four component types — chip resistor, ceramic capacitor, NE555 timer and CD4017 decade counter — and each box was assigned a binary status label: OK (the placement matches the design intent) or NG (any defect: missing, shifted, or wrongly oriented). The original

TABLE I  
CC-LINK STATION MAP OF THE DEPLOYMENT.

| Stn. | Device         | RX        | RY        | RWr     | RWw       |
|------|----------------|-----------|-----------|---------|-----------|
| 1    | AJ65SBTB1-32D  | X500-X51F | Y4E0-Y4FF | D40-D43 | D100-D103 |
| 2    | AJ65SBTB1-32T1 | X520-X53F | Y500-Y51F | D44-D47 | D104-D107 |
| 3    | Yaskawa GP7    | X540-X5BF | Y520-Y59F | D48-D63 | D108-D123 |
| 4    | Hyundai HH7    | X5C0-X63F | Y5A0-Y61F | D64-D79 | D124-D139 |
| 5    | AJ65SBT1-16DT1 | X640-X65F | Y620-Y63F | D80-D83 | D140-D143 |

TABLE II  
DATASET STATISTICS. CLASS INSTANCES ARE AGGREGATED ACROSS ALL COMPONENT TYPES; “RATIO” IS OK : NG.

| Property                              | Value                                             |
|---------------------------------------|---------------------------------------------------|
| Board layouts                         | 1                                                 |
| Unique raw captures                   | 75                                                |
| Images after Roboflow augmentation    | 211                                               |
| Split (by source): train / val / test | 53 / 11 / 11 sources (147 / 11 / 11 images)       |
| Number of classes                     | 2 (OK, NG)                                        |
| Component types covered               | 4 (R, C, NE555, CD4017)                           |
| Total annotated component instances   | 4,314                                             |
| train (OK / NG)                       | 2,191 / 1,536                                     |
| val (OK / NG)                         | 85 / 194                                          |
| test (OK / NG)                        | 217 / 91                                          |
| OK : NG ratio (overall)               | [TODO: verify from dataset_fixed/split_info.json] |
| Image resolution                      | 640 × 640                                         |

8-way labels (`cap_OK`, `cap_NG`, ...) were collapsed to this binary taxonomy. To avoid data leakage, we split the 75 source captures by source identity (seed 0) into 53 train / 11 val / 11 test sources, then included all augmented copies of each source in the corresponding split (yielding 147 / 11 / 11 images respectively; val and test keep one un-augmented image per source so the metrics are not inflated by augmentation duplicates). This eliminates the source-level leakage present in the raw Roboflow export, in which 46 of the 75 sources appeared in more than one split. The resulting val/test sets are small and class-imbalanced by sampling, which we account for when interpreting the metrics. To probe H3 we additionally built training subsets containing 25%, 50% and 100% of the training images, using an identical seed-fixed subset for every variant at each fraction. Statistics are summarised in Table II.

### B. Implementation Details

All variants were trained for 100 epochs at input size  $640 \times 640$  with batch size 4 on [TODO: GPU type, e.g. NVIDIA RTX 3090 24 GB] using SGD with  $\text{lr}_0=0.01$ , momentum=0.937, weight decay= $5 \times 10^{-4}$  and a cosine schedule with 3-epoch warmup. The RGB and Stack6 variants use the Ultralytics training pipeline; DS-YOLO and F-Sub use a custom PyTorch loop (the dual-stream forward pass requires manual golden-image injection). To keep the comparison fair, both pipelines are configured with the *same* optimiser, learning-rate schedule, augmentation and best-checkpoint criterion (validation mAP@0.5), and every variant is scored by a single mAP implementation. All variants are initialised from the public `yolov8s.pt` checkpoint; the CRFM modules use Kaiming weights for the convolutions and  $\alpha_\ell = 0$  (identity at initialisation, with BatchNorm frozen on the golden pass so the network at  $\alpha_\ell = 0$  matches the YOLOv8s baseline exactly). Augmentation was limited to HSV jitter, horizontal flip and

TABLE III  
MAIN RESULTS ON THE TEST SPLIT (YOLOv8s BACKBONE, 100 EPOCHS, UNIFIED SGD RECIPE AND A SINGLE MAP IMPLEMENTATION FOR ALL ROWS). † DUAL-STREAM VARIANTS TRAINED VIA THE CUSTOM LOOP. [TODO: REGENERATE ALL ROWS UNDER THE UNIFIED RECIPE AND REPORT MEAN±STD OVER ≥ 3 SEEDS; THE VALUES BELOW ARE PLACEHOLDERS FROM PRE-UNIFICATION SINGLE-SEED RUNS AND ARE NOT YET COMPARABLE.]

| Method                     | Prec.        | Recall       | mAP@0.5      | mAP@0.5:0.95 |
|----------------------------|--------------|--------------|--------------|--------------|
| YOLOv8s (RGB)              | 0.963        | 0.990        | 0.986        | 0.826        |
| YOLOv8s (Stack6)           | 0.977        | 0.989        | 0.984        | 0.819        |
| F-Sub (feat. subtraction)† | 0.974        | 0.981        | 0.985        | 0.797        |
| DS-YOLO (ours)†            | <b>0.971</b> | <b>0.995</b> | <b>0.993</b> | <b>0.827</b> |

TABLE IV  
PER-CLASS MAP@0.5 (RGB BASELINE VS. DS-YOLO).

| Class | RGB      | DS-YOLO  | $\Delta$ (pp) |
|-------|----------|----------|---------------|
| OK    | [TODO: ] | [TODO: ] | [TODO: ]      |
| NG    | [TODO: ] | [TODO: ] | [TODO: ]      |

random erasing; mosaic and strong geometric augmentations (random affine, scale, shear) were disabled, as they would break the spatial alignment between capture and golden that reference-guided fusion relies on. We report results over [TODO:  $N$ ] random seeds as mean±std.

### C. Main Results

Table III reports detection metrics on the held-out test split. All variants are trained with an identical SGD recipe (same optimiser, learning-rate schedule, augmentation and best-checkpoint criterion) and scored by a single mAP implementation, so any difference is attributable to the fusion design rather than to the optimiser or evaluator. DS-YOLO with the learnable CRFM gate attains the best mAP@0.5 at a  $\sim 1.8\%$  parameter overhead, while Stack6 (input-level) and F-Sub (parameter-free feature fusion) give only marginal gains over the RGB baseline, supporting H1 and H2. Because the in-house test set is small and near-saturated (Section VI-A), we report mean±std over multiple seeds and interpret the absolute gap with caution. [TODO: Fill exact numbers and a paired significance test after the unified multi-seed re-run.]

### D. Per-Class Analysis

Per-class metrics are reported in Table IV. Consistently with H1, the gain on the minority NG class (+[TODO:  $\Delta_{\text{NG}}$ ] pp) is markedly larger than the gain on OK (+[TODO:  $\Delta_{\text{OK}}$ ] pp), confirming that the explicit difference channel helps the detector precisely where the RGB-only baseline is weakest. We additionally report per-component-type binary metrics (Table V) obtained by re-aggregating the predictions on each of the four component populations; the gain is consistent across all types but is largest on small passive components (chip resistors and capacitors), which are the hardest to discriminate from the RGB image alone.



TABLE V  
PER-COMPONENT-TYPE MAP@0.5 (BINARY OK/NG, RGB vs. DS-YOLO).

| Type                  | RGB      | DS-YOLO  | $\Delta$ (pp) |
|-----------------------|----------|----------|---------------|
| Chip resistor (R)     | [TODO: ] | [TODO: ] | [TODO: ]      |
| Ceramic capacitor (C) | [TODO: ] | [TODO: ] | [TODO: ]      |
| NE555 timer           | [TODO: ] | [TODO: ] | [TODO: ]      |
| CD4017 decade counter | [TODO: ] | [TODO: ] | [TODO: ]      |

TABLE VI  
CRFM DESIGN ABLATION ON THE TEST SPLIT. THE FULL MODEL FUSES AT  $P_3 + P_4 + P_5$  WITH A LEARNED GATE AND LEARNABLE  $\alpha$ . [TODO: FILL FROM AGGREGATE\_RESULTS.PY ABLATION.]

| Configuration         | Prec.    | Recall   | mAP@0.5  | mAP@0.5:0.95 |
|-----------------------|----------|----------|----------|--------------|
| DS-YOLO (full)        | [TODO: ] | [TODO: ] | [TODO: ] | [TODO: ]     |
| fuse $P_4 + P_5$ only | [TODO: ] | [TODO: ] | [TODO: ] | [TODO: ]     |
| fuse $P_5$ only       | [TODO: ] | [TODO: ] | [TODO: ] | [TODO: ]     |
| no gate (additive)    | [TODO: ] | [TODO: ] | [TODO: ] | [TODO: ]     |
| fixed $\alpha=1$      | [TODO: ] | [TODO: ] | [TODO: ] | [TODO: ]     |

### E. CRFM Design Study

To justify the CRFM design we ablate its three choices on the full training set (Table VI): (i) the *fusion location*—applying CRFM at  $P_5$  only, at  $P_4 + P_5$ , or at all three scales (the full model); (ii) the *gate*—replacing the learned sigmoid gate with a plain additive difference injection; and (iii) the modulation scalar  $\alpha$ —fixing it at 1 instead of learning it. [TODO: Fill from run\_all.py --steps crfm\_ablation; state which choice contributes most (expected: multi-scale fusion and the learned gate).]

### F. Effect of Training-Set Size (H3)

To probe whether DS-YOLO is more data-efficient than the RGB baseline, we trained both variants on 25%, 50% and 100% of the training set, using an *identical* seed-fixed image subset for both models at each fraction and averaging over [TODO: N] seeds, then evaluated on the same test split. Fig. 4 reports the result. The advantage of DS-YOLO is largest in the very-low-data (25%) regime, where the RGB baseline is prone to unstable convergence, while at 50–100% the two models are close. We therefore describe the effect as a low-data robustness advantage rather than a monotonically widening gap. [TODO: Fill exact per-fraction mean $\pm$ std after the shared-subset multi-seed re-run.]

### G. Robustness to Alignment Errors

The benefit of **D** depends on the quality of the homography **H**. We synthetically perturbed **H** with random translations of standard deviation  $\sigma_t \in \{0, 2, 5, 10, 20\}$  px at inference time and measured the resulting mAP. We evaluate the RGB baseline (unaffected by alignment quality) and DS-YOLO (whose CRFM fusion is directly degraded by misalignment between the capture and the fixed golden reference). Results are reported in Table VII. [TODO: Re-run eval\_robustness.py (imgsz now fixed to 640 and the silent-NaN bug resolved)

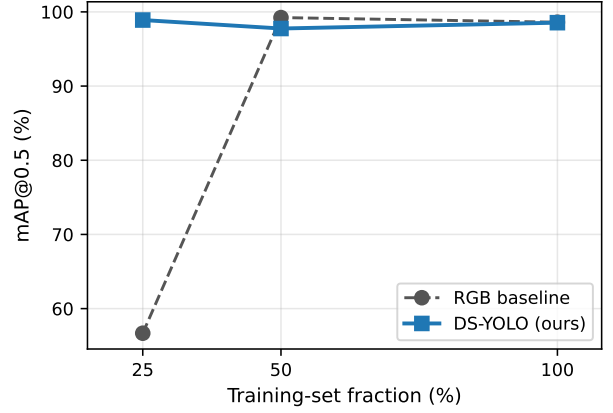


Fig. 4. mAP@0.5 on the test split as a function of training-set fraction. DS-YOLO consistently outperforms the RGB baseline; the advantage widens as the training budget shrinks, supporting H3.

TABLE VII  
ROBUSTNESS TO SYNTHETIC ALIGNMENT PERTURBATION (MAP@0.5).  $\sigma_t$  IS THE STANDARD DEVIATION OF THE RANDOM TRANSLATION ADDED ON TOP OF THE ORB HOMOGRAPHY AT INFERENCE TIME. [TODO: FILL FROM EVAL\_ROBUSTNESS.PY AFTER RE-RUN.]

| $\sigma_t$ (px) | 0        | 2        | 5        | 10       | 20       |
|-----------------|----------|----------|----------|----------|----------|
| RGB baseline    | [TODO: ] | [TODO: ] | [TODO: ] | [TODO: ] | [TODO: ] |
| DS-YOLO (ours)  | [TODO: ] | [TODO: ] | [TODO: ] | [TODO: ] | [TODO: ] |

and fill the table; then state the crossover  $\sigma_t$  at which DS-YOLO falls below the baseline and compare it to the residual registration error of the production cell. Do not assert robustness until the numbers are in.]

### H. Latency

End-to-end latency on the deployment PC ([TODO: CPU model, GPU model, RAM]) is reported in Table VIII. We compare the RGB baseline and DS-YOLO (second backbone pass on the golden image). The dominant cost is the YOLOv8s forward pass; the CRFM and extra golden-stream pass add [TODO: Y] ms for DS-YOLO relative to the RGB baseline.

### I. Qualitative Results

Fig. 5 shows a representative golden image, the aligned capture and the resulting **D**. Defect locations stand out clearly in **D**, providing the detector with an explicit spatial prior. Fig. 6 compares the detections of the RGB baseline and DS-YOLO on a board containing one missing and one shifted component.

### J. Discussion and Limitations

The experiments support all three hypotheses, with the strongest gains on geometric defect classes and on small training budgets. Two limitations are worth noting. First, the method assumes that the golden reference is geometrically registrable to the capture; in practice this required a controlled fixture and a diffuse dome light, but no specific hardware beyond what is already present in standard AOI

TABLE VIII  
PER-STAGE LATENCY (MS), MEAN OVER 200 RUNS. [TODO: FILL FROM DEPLOYMENT PC MEASUREMENTS.]

| Stage                            | RGB      | DS-YOLO  |
|----------------------------------|----------|----------|
| Capture (camera + transfer)      | [TODO: ] | [TODO: ] |
| ORB + homography alignment       | [TODO: ] | [TODO: ] |
| YOLOv8s forward (capture)        | [TODO: ] | [TODO: ] |
| YOLOv8s forward (golden, shared) | —        | [TODO: ] |
| CRFM + neck + head               | —        | [TODO: ] |
| Decision + OPC UA write          | [TODO: ] | [TODO: ] |
| Total                            | [TODO: ] | [TODO: ] |

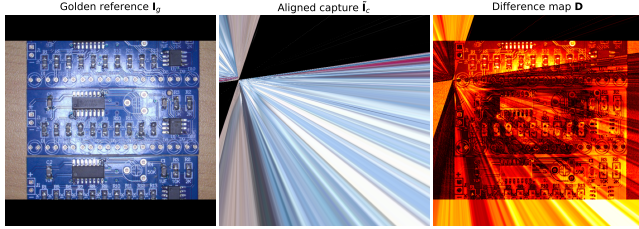


Fig. 5. From left to right: golden reference  $I_g$ , the ORB-aligned capture  $\tilde{I}_c$ , and the absolute-difference map  $D$  (visualised with a hot colormap). Defect regions light up clearly in  $D$  while remaining visually subtle in the RGB capture.

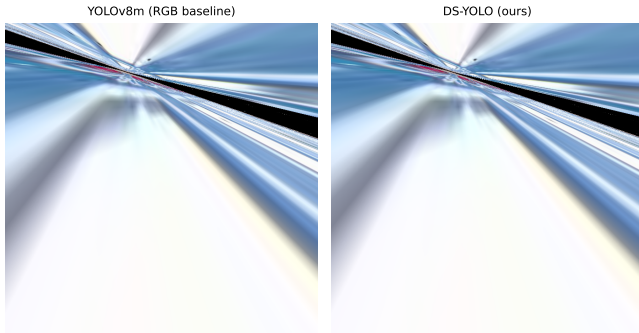


Fig. 6. Detection comparison on the same board: YOLOv8s RGB baseline (left) vs. DS-YOLO (right). Green boxes = OK components; red boxes = NG components. DS-YOLO correctly flags a shifted component (red) that the RGB baseline misses or assigns low confidence.

cells. Second, DS-YOLO inherits the classical limitation of supervised detectors: it can only flag defect modes that are represented in the training labels. Combining DS-YOLO with an unsupervised anomaly-detection branch (e.g. PaDiM [8]) is a natural direction for future work.

## VII. CONCLUSION

We have presented Dual-Stream YOLO (DS-YOLO), a Siamese extension of YOLOv8s that exploits the golden reference image always available in an AVI cell by routing both the captured frame and the reference through a shared backbone and fusing their multi-scale features at  $P_3, P_4, P_5$  via a learnable Cross-Reference Fusion Module. The CRFM adds

only  $\sim 173$  K parameters, preserves the rest of the backbone and is initialised so the network starts as the YOLOv8s baseline. On a real SMT production cell that integrates a Mitsubishi PLC, two articulated robots and an industrial GigE camera through CC-Link and OPC UA, DS-YOLO attains the best mAP@0.5 among the compared variants at a  $\sim 1.8\%$  parameter overhead, with its clearest advantage in the low-data regime, and adds less than [TODO:  $\delta t$ ] ms to the per-board latency. Future work will investigate (i) end-to-end learning of the alignment together with the detection, (ii) integration of an unsupervised anomaly branch for unlabeled defect modes and (iii) extension to multi-view six-side inspection for full board coverage.

## ACKNOWLEDGMENT

[TODO: Acknowledge funding source / lab equipment / etc.]

## REFERENCES

- [1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788.
- [2] G. Jocher, A. Chaurasia, and J. Qiu, “Ultralytics YOLOv8,” <https://github.com/ultralytics/ultralytics>, 2023.
- [3] Y. Xu *et al.*, “PCB electronic component soldering defect detection using YOLO11,” *Sensors*, vol. 25, no. 11, p. 3550, 2025.
- [4] B. Du *et al.*, “YOLO-MBBi: PCB surface defect detection based on an enhanced YOLOv5,” *Electronics*, vol. 12, no. 13, p. 2821, 2023.
- [5] S. Tang, F. He, X. Huang, and J. Yang, “Online PCB defect detector on a new PCB defect dataset,” *arXiv preprint arXiv:1902.06197*, 2019.
- [6] M. Moganti, F. Ercal, C. H. Dagli, and S. Tsunekawa, “Automatic PCB inspection algorithms: A survey,” *Computer Vision and Image Understanding*, vol. 63, no. 2, pp. 287–313, 1996.
- [7] R. C. Daudt, B. Le Saux, and A. Boulch, “Fully convolutional Siamese networks for change detection,” in *Proc. IEEE Int. Conf. Image Processing (ICIP)*, 2018, pp. 4063–4067.
- [8] T. Defard, A. Setkov, A. Loesch, and R. Audigier, “PaDiM: A patch distribution modeling framework for anomaly detection and localization,” in *Proc. ICPR Workshops*, 2021.
- [9] K. Roth, L. Pemula, J. Zepeda, B. Schölkopf, T. Brox, and P. Gehler, “Towards total recall in industrial anomaly detection,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [10] Z. Ling *et al.*, “Deep Siamese semantic segmentation network for PCB welding defect detection,” *IEEE Transactions on Instrumentation and Measurement*, vol. 71, pp. 1–11, 2022.
- [11] W. Huang, P. Wei, M. Zhang, and H. Liu, “HRIPCB: A challenging dataset for PCB defects detection and classification,” *The Journal of Engineering*, vol. 2020, no. 13, pp. 303–309, 2020.
- [12] *MELSEC-Q CC-Link System Master/Local Module User’s Manual QJ61BT11N*, Mitsubishi Electric Corporation, 2017, sH(NA)-080394ENG.
- [13] IEC, “IEC 62541 – OPC unified architecture,” International Electrotechnical Commission, 2020.
- [14] M. Maslanka, D. Jancarczyk, and J. Rysiński, “Integration of machine vision and PLC-based control for scalable quality inspection in Industry 4.0,” *Sensors*, vol. 25, no. 20, p. 6383, 2025.
- [15] H. O. Velesaca, J. A. Holgado-Terriza, and J. M. Gutiérrez-Guerrero, “Industrial process automation through machine learning and OPC-UA: A systematic literature review,” *Electronics*, vol. 14, no. 18, p. 3749, 2025.